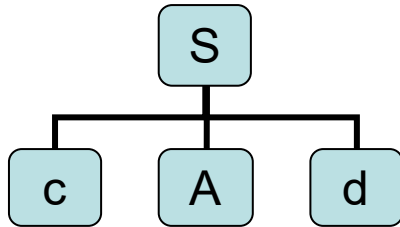


Top-down parsing

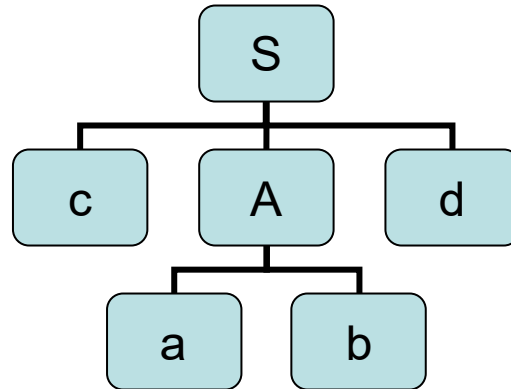
- Types of parsers:
 - Top-down:
 - repeatedly rewrite the start symbol;
 - find a left-most derivation of the input string;
 - easy to implement;
 - Bottom-up:
 - start with tokens and combine them to form interior nodes of the parse tree;
 - find a right-most derivation of the input string;
 - accept when the start symbol is reached;
 - it is more prevalent.
- Both can not parse all context-free grammars

Top-down parsing with backtracking

- $S \rightarrow cAd$
- $A \rightarrow ab$
- $A \rightarrow a$
- $w = cad$

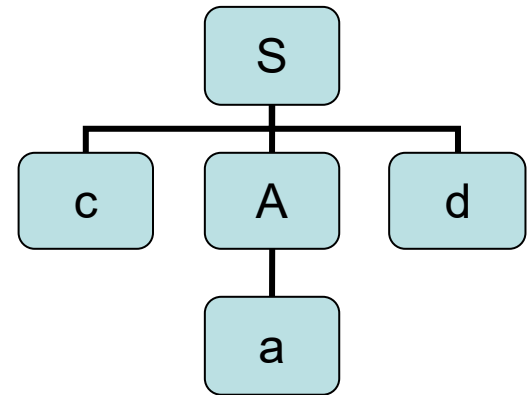


cad
↑



cad
↑

cad
↑



cad
↑

Parsing trace

Expansion	Remaining input	Action
S	cad	Try $S \rightarrow cAd$
cAd	cad	Match c
Ad	ad	Try $A \rightarrow ab$
abd	ad	Match a
bd	d	Dead end, backtrack
ad	ad	Try $A \rightarrow a$
ad	ad	Match a
d	d	Match d
Success		

$S \rightarrow cAd$
 $A \rightarrow ab|a$

Parsing trace

Expansion	Remaining input	Action
S	cad	Try $S \rightarrow cAd$
cAd	cad	Match c
Ad	ad	Try $A \rightarrow ab$
abd	ad	Match a
bd	d	Dead end, backtrack
ad	ad	Try $A \rightarrow a$
ad	ad	Match a
d	d	Match d
Success		

- $S \rightarrow cAd$
- $A \rightarrow ab|a$

Another example

$S \rightarrow AB$
 $A \rightarrow aA \mid \epsilon$
 $B \rightarrow bB \mid b$

Expansion	Remaining input	Action
S	aabb	Try $S \rightarrow AB$
AB	aabb	Try $A \rightarrow aA$
aAB	aabb	match a
AB	abb	$A \rightarrow aA$
aAB	abb	match a
AB	bb	$A \rightarrow \text{epsilon}$
B	bb	$B \rightarrow bB$
bB	bb	match b
B	b	$B \rightarrow b$
b	b	match

$S \rightarrow AB$
 $\rightarrow aAB$
 $\rightarrow aaAB$
 $\rightarrow aa\epsilon B$
 $\rightarrow aabB$
 $\rightarrow aabb$

Top down vs. bottom up parsing

- Given the rules

- $S \rightarrow AB$
- $A \rightarrow aA | \epsilon$
- $B \rightarrow bB | b$

- How to parse aabb ?

- Topdown approach

$S \rightarrow AB$
 $\rightarrow aAB$
 $\rightarrow aaAB$
 $\rightarrow aa\epsilon B$
 $\rightarrow aabB$
 $\rightarrow aabb$

Note that it is a left most derivation

- Bottom up approach

aabb
 $\leftarrow a\ abb$
 $\leftarrow aa\ bb$
 $\leftarrow aa\epsilon\ bb$
 $\leftarrow aaA\ bb$
 $\leftarrow aA\ bb$
 $\leftarrow A\ bb$
 $\leftarrow Ab\ b$
 $\leftarrow Abb$
 $\leftarrow AbB$
 $\leftarrow AB$
 $\leftarrow S$

- If read backwards, the derivation is right most
- In both topdown and bottom up approaches, the input is scanned from left to right

Recursive descent parsing

- Each method corresponds to a non-terminal

```
static boolean checkS() {  
    int savedPointer = pointer;  
    if (checkA() && checkB())  
        return true;  
    pointer = savedPointer;  
    return false;  
}
```

$S \rightarrow AB$

```
static boolean checkA() {  
    int savedPointer = pointer;  
    if (nextToken().equals('a') && checkA())  
        return true;  
    pointer = savedPointer;  
    return true;  
}
```

$A \rightarrow aA | \epsilon$

//B→bB|b

```
static boolean checkB() {  
    int savedPointer = pointer;  
    if (nextToken().equals('b') && checkB())  
        return true;  
    pointer = savedPointer;  
    if(nextToken().equals('b')) return true;  
    pointer = savedPointer;  
    return false;  
}
```


Left recursion

- What if the grammar is changed to

$S \rightarrow AB$

$A \rightarrow Aa | \epsilon$

$B \rightarrow b | bB$

- The corresponding methods are

```
static boolean checkA() {
    int savedPointer = pointer;
    if (checkA() && nextToken().equals('a'))
        return true;
    pointer = savedPointer;
    return true;
}
static boolean checkB() {
    int savedPointer = pointer;
    if (nextToken().equals('b'))
        return true;
    pointer = savedPointer;
    if (nextToken().equals('b') && checkB()) return true;
    return false;
    pointer = savedPointer;
}
```

Recursive descent parsing (a complete example)

- Grammar

program $\rightarrow$ statement program | statement

statement $\rightarrow$ assignment

assignment $\rightarrow$ ID EQUAL expr

... ..

- Task:

- Write a java program that can judge whether a program is syntactically correct.
- This time we will write the parser manually.
- We can use the scanner

- How to do it?

RecursiveDescent.java outline

```
1. static int pointer=-1;
2. static ArrayList tokens=new ArrayList();
3. static Symbol nextToken() { }
4. public static void main(String[] args) {
5.     Calc3Scanner scanner=new
6.         Calc3Scanner(new FileReader("calc2.input"));
7.     Symbol token;
8.     while(token=scanner.yylex().sym!=Calc2Symbol.EOF)
9.         tokens.add(token);
10.    boolean legal= program() && nextToken()==null;
11.    System.out.println(legal);
12.}

13. static boolean program() throws Exception {...}
14. static boolean statement() throws Exception {...}
15. static boolean assignment () throws Exception {...}
16. static boolean expr() {...}
```

One of the methods

```
1. /**  program-->statement program
2.      program-->statement
3. */
4. static boolean program() throws Exception {
5.     int savedPointer = pointer;
6.     if (statement() && program()) return true;
7.     pointer = savedPointer;
8.     if (statement()) return true;
9.     pointer = savedPointer;
10.    return false;
11. }
```

Recursive Descent parsing

- Recursive descent parsing is an easy, natural way to code top-down parsers.
 - All non terminals become procedure calls that return true or false;
 - all terminals become matches against the input stream.
- Example:

```
/** assignment--> ID=exp */
static boolean assignment() throws Exception{
    int savePointer= pointer;
    if (  nextToken().sym==Calc2Symbol.ID
        && nextToken().sym==Calc2Symbol.EQUAL
        && expr() )
        return true;
    pointer = savePointer;
    return false;
}
```

Summary of recursive descent parser

- Simple enough that it can easily be constructed by hand;
- Not efficient;
- Limitations:

```
/** E → E+T | T */  
static boolean expr() throws Exception {  
    int savePointer = pointer;  
    if ( expr()  
        && nextToken().sym==Calc2Symbol.PLUS  
        && term())  
        return true;  
    pointer = savePointer;  
    if (term()) return true;  
    pointer = savePointer;  
    return false;  
}
```

- A recursive descent parser can enter into infinite loop.

Left recursion

- Definition

- A grammar is left recursive if it has a nonterminal A such that there is a derivation $A \Rightarrow^+ A\alpha$
- There are two kinds of left recursion:
 - direct left recursive : $A \Rightarrow A\alpha$
 - in-direct left-recursive: $A \Rightarrow^+ A\alpha$, but not $A \Rightarrow A\alpha$

- Example:

$E \rightarrow E+T \mid T$ is direct left recursive

- Indirect left-recursive

$S \rightarrow Aa \mid b$

$A \rightarrow Ac \mid Sd \mid \epsilon$

Is S left recursive?

$S \Rightarrow Aa$

$\Rightarrow Sda$

$S \Rightarrow^+ Sda$

Left recursion has to be removed for recursive descent parser

Look at the previous example that works:

$E \rightarrow T + E \mid T$

```
static boolean expr() throws Exception {  
    int savePointer = pointer;  
    if (term() && nextToken().sym == Calc2Symbol.PLUS && expr()) return true;  
    pointer = savePointer;  
    if (term()) return true;  
    pointer = savePointer;  
    return false;  
}
```

What if the grammar is left recursive?

$E \rightarrow E + T \mid T$

```
static boolean expr() throws Exception {  
    int savePointer = pointer;  
    if (expr() && nextToken().sym == Calc2Symbol.PLUS && term ()) return true;  
    pointer = savePointer;  
    if (term()) return true;  
    pointer = savePointer;  
    return false;  
}
```

There will be infinite loop!

Remove left recursion

- Direct left recursion

$$A \rightarrow A\alpha | \beta$$

expanded form: $A \rightarrow \beta \alpha \alpha \dots \alpha$

Left recursion removed:

$$A \rightarrow \beta Z$$

$$Z \rightarrow \alpha Z | \epsilon$$

- Example:

$$E \rightarrow E+T | T$$

expanded form: $E \rightarrow T +T +T \dots +T$

$$E \rightarrow TZ$$

$$Z \rightarrow +TZ | \epsilon$$

Remove left recursion

- In general, for a production

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

where no β_i begins with A .

It can be replaced by:

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$$

Predictive parsing

- Predictive parser is a special case of top-down parsing when no backtracking is required;
- At each non-terminal node, the action to undertake is unambiguous;

STAT $\rightarrow$ if ...

 | while ...

 | for ...

- Not general enough to handle real programming languages;
- Grammar must be left factored;

IFSTAT $\rightarrow$ if EXPR then STAT

 | if EXPR then STAT else STAT

- A predictive parser must choose the correct version of the IFSTAT before seeing the entire input
- The solution is to factor out common terms:

IFSTAT $\rightarrow$ if EXPR then STAT IFREST

IFREST $\rightarrow$ else STAT | ϵ

- Consider another familiar example:

$E \rightarrow T + E \mid T$

Left factoring

- General method

For a production $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \alpha \beta_n \mid \gamma$

where γ represents all alternatives that do not begin with α ,
it can be replaced by

$$A \rightarrow \alpha B \mid \gamma$$
$$B \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

- Example

$$E \rightarrow T + E \mid T$$

Can be transformed into:

$$E \rightarrow T E'$$
$$E' \rightarrow +E \mid \varepsilon$$

Predictive parsing

- The recursive descent parser is not efficient because of the backtracking and recursive calls.
- *a predictive parser* does not require backtracking.
 - able to choose the production to apply solely on the basis of the next input symbol and the current nonterminal being processed
- To enable this, the grammar must be $LL(1)$.
 - The first “L” means we scan the input from left to right;
 - the second “L” means we create a leftmost derivation;
 - the 1 means one input symbol of lookahead.

LL(1) Grammar

- LL(1) grammar has no left-recursive productions and has been left factored.
 - left factored grammar with no left recursion may not be LL(1)
- There are grammars that cannot be modified to become LL(1).
- In such cases, another parsing technique must be employed, or special rules must be embedded into the predictive parser.

First() set--motivation

- Navigating through two choices seemed simple enough, however, what happens where we have many alternatives on the right side?
 - `statement` $\rightarrow$ `assignment` | `returnStatement` | `ifStatement` | `whileStatement` | `blockStatement`
- When implementing the `statement()` method, how are we going to be able to determine which of the 5 options to match for any given input?
- Remember, we are trying to do this without backtracking, and just one token of lookahead, so we have to be able to make immediate decision with minimal information— this can be a challenge!
- Fortunately, many production rules starts with terminals, which can help in deciding which rule to use.
 - For example, if the input token is 'while', the program should know that the `whileStatement` rule will be used.

Fisrt(): motivating example

- On many cases, rules starts with non-terminals

$S \rightarrow Ab|Bc$

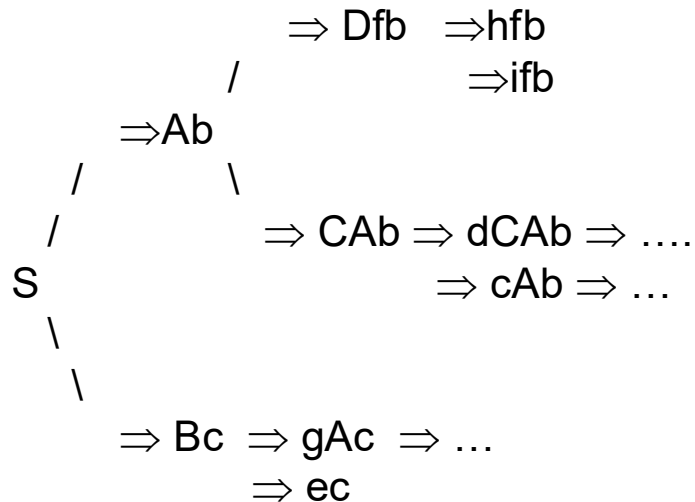
$A \rightarrow Df|CA$

$B \rightarrow gA|e$

$C \rightarrow dC|c$

$D \rightarrow h|i$

How to parse “gchfc”?



$S \Rightarrow Bc$
 $\Rightarrow gAc$
 $\Rightarrow gCAc$
 $\Rightarrow gcAc$
 $\Rightarrow gcDfc$
 $\Rightarrow gchfc$

if the next token is h, i, d, or c,
alternative Ab should be
selected.

If the next token is g or e,
alternative Bc should be
selected.

In this way, by looking at the
next token, the parser is
able to decide which rule
to use without exhaustive
searching.

First(): Definition

- The *First set* of a sequence of symbols α , written as $\text{First}(\alpha)$, is the set of terminals which start the sequences of symbols derivable from α .
 - If $\alpha \Rightarrow^* a\beta$, then a is in $\text{First}(\alpha)$.
 - If $\alpha \Rightarrow^* \varepsilon$, then ε is in $\text{First}(\alpha)$.
- Given a production with a number of alternatives:
 - $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots$,
 - we can write a predicative parser only if all the sets $\text{First}(\alpha_i)$ are disjoint.

First() algorithm

- First(): compute the set of terminals that can begin a rule
 - if **a** is a terminal, then first(a) is {a}.
 - if A is a non-terminal and $A \rightarrow a\alpha$ is a production, then add **a** to first(A).
if $A \rightarrow \epsilon$ is a production, add ϵ to first(A).
 - if $A \rightarrow \alpha_1 \alpha_2 \dots \alpha_m$ is a production, add First(α_1)- ϵ to First(A).
If α_1 can derive ϵ , add First(α_2)- ϵ to First(A).
If both α_1 and α_2 derives ϵ , add First(α_3)- ϵ to First(A). and so on.
If $\alpha_1 \alpha_2 \dots \alpha_m \Rightarrow^* \epsilon$, add ϵ to First(A).

- Example

$S \rightarrow Aa \mid b$

$A \rightarrow bdZ \mid eZ$

$Z \rightarrow cZ \mid adZ \mid \epsilon$

$\text{First}(A) = \{\text{First}(b), \text{First}(e)\} = \{b, e\}$

(by rule 2, rule 1)

$\text{First}(Z) = \{a, c, \epsilon\}$

(by rule 2, rule 1)

$\text{First}(S)$

$= \{\text{First}(A), \text{First}(b)\}$ (by rule 3)

$= \{\text{First}(A), b\}$ (by rule 1)

$= \{b, e, b\} = \{b, e\}$ (by rule 2)

A slightly modified example

$$S \rightarrow Aa \mid b$$

$$A \rightarrow bdZ \mid eZ \mid \epsilon$$

$$Z \rightarrow cZ \mid adZ \mid \epsilon$$

$$\text{First}(S)$$

$$= \{\text{First}(A), \text{First}(b)\} \quad (\text{by rule 3})$$

$$= \{\text{First}(A), b\} \quad (\text{by rule 1})$$

$$= \{b, e, b\} = \{b, e\} \quad (\text{by rule 2}) ?$$

$$\text{First}(S) = \{\text{First}(A), \text{First}(a), b\} = \{a, b, e, \epsilon\} \quad ?$$

$$\text{Answer: } \text{First}(S) = \{a, b, e\}$$

Follow()– motivation

- Consider
 - $S \rightarrow^* aaAb$
 - Where $A \rightarrow \epsilon | aA$.
- When can $A \rightarrow \epsilon$ be used? What is the next token expected?
- In general, when A is nullable, what is the next token we expect to see?
 - A non-terminal A is nullable if ϵ in $\text{First}(A)$, or
 - $A \rightarrow^* \epsilon$
- the next token would be the first token of the symbol following A in the sentence being parsed.

Follow()

- Follow(): find the set of terminals that can immediately follow a non-terminal
 1. \$(end of input) is in Follow(S), where S is the start symbol;
 2. For the productions of the form $A \rightarrow \alpha B \beta$ then everything in First(β) but ϵ is in Follow(B).
 3. For productions of the form $A \rightarrow \alpha B$ or $A \rightarrow \alpha B \beta$ where First(β) contains ϵ , then everything in Follow(A) is in Follow(B).
 - $aAb \Rightarrow a\alpha Bb$

- Example

$S \rightarrow Aa \mid b$

$A \rightarrow bdZ \mid eZ$

$Z \rightarrow cZ \mid adZ \mid \epsilon$

Follow(S)

= { $\$$ } (by rule 1)

Follow(A)

= {a} (by rule 2)

Follow(Z)

= {Follow(A)} = {a} (by rule 3)

Compute First() and Follow()

- | | |
|-------------------------------------|--------------------------------|
| 1. $E \rightarrow TE'$ | First (E) |
| 2. $E' \rightarrow +TE' \epsilon$ | = First(T) |
| 3. $T \rightarrow FT'$ | = First(F) |
| 4. $T' \rightarrow *FT' \epsilon$ | = { (, id } |
| 5. $F \rightarrow (E) id$ | First (E') |
| | = { +, ϵ } |
| | First (T') = { *, ϵ } |

Follow (E)
= {), \$ }
= Follow(E')

Follow(T)
= Follow(T')
= { +,), \$ } First(E') except ϵ plus Follow(E)

Follow (F)
= { *, +,), \$ } First(T') except ϵ plus Follow(T')

The use of First() and Follow()

- If we want to expand S in this grammar:
 $S \rightarrow A \dots \mid B \dots$
 $A \rightarrow a \dots$
 $B \rightarrow b \dots \mid a \dots$
- If the next input character is b, we should rewrite S with A... or B?
 - since $\text{First}(B) = \{a, b\}$, and $\text{First}(A) = \{a\}$, we know to rewrite S with B;
 - First and Follow gives us information about the next characters expected in the grammar.
- If the next input character is a, how to rewrite S?
 - **a** is in both $\text{First}(A)$ and $\text{First}(B)$;
 - The grammar is not suitable for predictive parsing.

LL(1) parse table construction

- Construct a parse table (PT) with one axis the set of terminals, and the other the set of non-terminals.
- For all productions of the form $A \rightarrow \alpha$
 - Add $A \rightarrow \alpha$ to entry $PT[A, b]$ for each token b in $\text{First}(\alpha)$;
 - add $A \rightarrow \alpha$ to entry $PT[A, b]$ for each token b in $\text{Follow}(A)$ if $\text{First}(\alpha)$ contains ϵ ;
 - add $A \rightarrow \alpha$ to entry $PT[A, \$]$ if $\text{First}(\alpha)$ contains ϵ and $\text{Follow}(A)$ contains $\$$.

$S \rightarrow Aa \mid b$

$A \rightarrow b d Z \mid eZ$

$Z \rightarrow cZ \mid a d Z \mid \epsilon$

	a	b	c	d	e	\$
S		$S \rightarrow Aa$ $S \rightarrow b$			$S \rightarrow Aa$	
A		$A \rightarrow bdZ$			$A \rightarrow eZ$	
Z	$Z \rightarrow \epsilon$ $Z \rightarrow adZ$		$Z \rightarrow cZ$			

		First	Follow
S	Aa	b, e	\$
	b	b	
A	bdZ	b	a
	eZ	e	
Z	cZ	c	a
	adZ	a	
	ϵ	ϵ	

Construct the parsing table

- if $A \rightarrow \alpha$, which column we place $A \rightarrow \alpha$ in row A ?
 - in the column of t , if t can start a string derived from α , i.e., t in $\text{First}(\alpha)$.
 - what if α is empty? put $A \rightarrow \alpha$ in the column of t if t can follow an A , i.e., t in $\text{Follow}(A)$.

	a	b	c	d	e	\$
S		$S \rightarrow Aa$ $S \rightarrow b$			$S \rightarrow Aa$	
A		$A \rightarrow bdZ$			$A \rightarrow eZ$	
Z	$Z \rightarrow \epsilon$ $Z \rightarrow adZ$		$Z \rightarrow cZ$			

$S \rightarrow Aa \mid b$
 $A \rightarrow bdZ \mid eZ$
 $Z \rightarrow cZ \mid adZ \mid \epsilon$

Stack	Remaining Input	Action
S\$	bda\$	Predict $S \rightarrow Aa$ or $S \rightarrow b$? suppose Aa is used
Aa\$	bda\$	Predict $A \rightarrow bdZ$
bdZa\$	bda\$	match
dZa\$	da\$	match
Za\$	a\$	Predict $Z \rightarrow \epsilon$
a\$	a\$	match
\$	\$	accept

- Note that it is not LL(1) because there are more than one rule can be selected.
- The correspondent (leftmost) derivation
 $S \rightarrow Aa \rightarrow bdZa \rightarrow bd\epsilon a$
- Note when $Z \rightarrow \epsilon$ rule is used.

LL(1) grammar

- If the table entries are unique, the grammar is said to be LL(1):
 - Scan the input from **L**eft to right;
 - performing a **L**eftmost derivation.
- LL(1) grammars can have all their parser decisions made using one token look ahead.
- In principle, can have LL(k) parsers with $k > 1$.
- Properties of LL(1)
 - Ambiguous grammar is never LL(1);
 - Grammar with left recursion is never LL(1);
- A grammar G is LL(1) iff whenever $A \rightarrow \alpha \mid \beta$ are two distinct productions of G , the following conditions hold:
 - For no terminal a do both α and β derive strings beginning with a (i.e., First sets are disjoint);
 - At most one of α and β can derive the empty string
 - If $\beta \Rightarrow^* \varepsilon$ then α does not derive any string beginning with a terminal in $\text{Follow}(A)$

A complete example for LL(1) parsing

$S \rightarrow P$

$P \rightarrow \{ D; C \}$

$D \rightarrow d, D \mid d$

$C \rightarrow c, C \mid c$

- The above grammar corresponds loosely to the structure of programs. (program is a sequence of declarations followed by a sequence of commands).
- Need to left factor the grammar first.

$S \rightarrow P$

$P \rightarrow \{ D; C \}$

$D \rightarrow d D_2$

$D_2 \rightarrow , D \mid \epsilon$

$C \rightarrow c C_2$

$C_2 \rightarrow , C \mid \epsilon$

	First	Follow
S	{	\$
P	{	\$
D	d	;
D ₂	, ϵ	;
C	c	}
C ₂	, ϵ	}

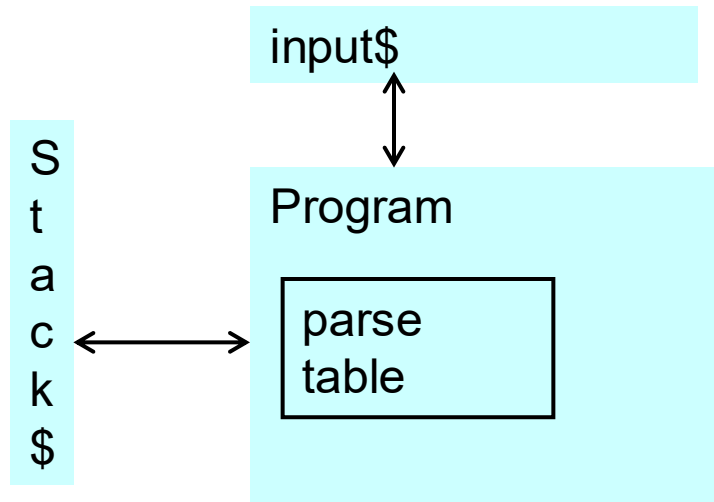
Construct LL(1) parse table

	{	}	;	,	c	d	\$
S	$S \rightarrow P\$$						
P	$P \rightarrow \{D; C\}$						
D						$D \rightarrow dD_2$	
D ₂			$D_2 \rightarrow \epsilon$	$D_2 \rightarrow ,D$			
C					$C \rightarrow cC_2$		
C ₂		$C_2 \rightarrow \epsilon$		$C_2 \rightarrow ,C$			

$S \rightarrow P$
 $P \rightarrow \{ D; C \}$
 $D \rightarrow d D_2$
 $D_2 \rightarrow , D \mid \epsilon$
 $C \rightarrow c C_2$
 $C_2 \rightarrow , C \mid \epsilon$

	First	Follow
S	{	\$
P	{	\$
D	d	;
D ₂	, ϵ	;
C	c	}
C ₂	, ϵ	}

LL(1) parse program



- Stack: contain the current rewrite of the start symbol.
- Input: left to right scan of input.
- Parse table: contain the LL(k) parse table.

LL(1) parsing algorithm

- Use the stack, input, and parse table with the following rules:
 - *Accept*: if the symbol on the top of the stack is \$ and the input symbol is \$, successful parse
 - *match*: if the symbol on the top of the stack is the same as the next input token, pop the stack and advance the input
 - *predict*: if the top of the stack is a non-terminal M and the next input token is **a**, remove M from the stack and push entry PT[M,a] on to the stack in reverse order
 - *Error*: Anything else is a syntax error

Running LL(1) parser

Stack	Remaining Input	Action
S	{d,d;c}\$	predict $S \rightarrow P\$$
P\$	{d,d;c}\$	predict $P \rightarrow \{D;C\}$
{ D ; C } \$	{d,d;c}\$	match {
D ; C } \$	d,d;c}\$	predict $D \rightarrow d D2$
d D2 ; C } \$	d,d;c}\$	match d
D2 ; C } \$	,d;c}\$	predict $D2 \rightarrow ,D$
, D ; C } \$	,d;c}\$	match ,
D ; C } \$	d;c}\$	predict $D \rightarrow d D2$
d D2 ; C } \$	d;c}\$	match d
D2 ; C } \$	;c}\$	predict $D2 \rightarrow \epsilon$
ϵ ; C } \$	;c} \$	match ;
C } \$	c}\$	predict $C \rightarrow c C2$
c C2 } \$	c}\$	match c
C2 } \$	}\$	predict $C2 \rightarrow \epsilon$
} \$	}\$	match }
\$	\$	accept

$S \rightarrow P$
 $P \rightarrow \{ D; C \}$
 $D \rightarrow d D2$
 $D2 \rightarrow , D \mid \epsilon$
 $C \rightarrow c C2$
 $C2 \rightarrow , C \mid \epsilon$

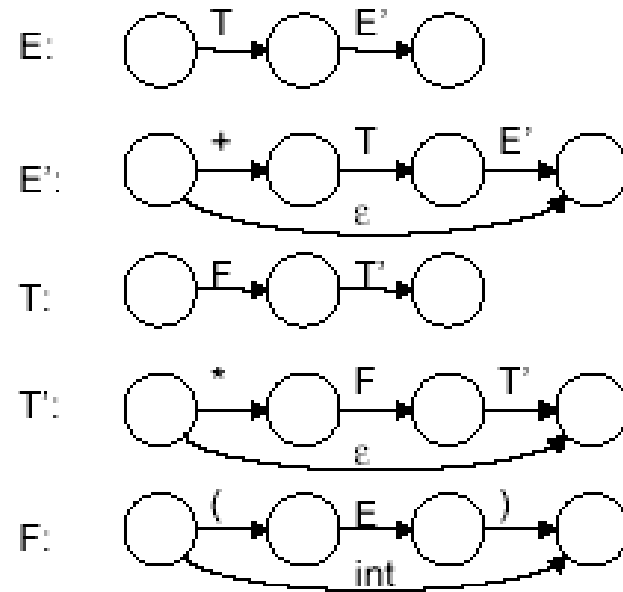
Derivation

$S \rightarrow P\$$
 $\rightarrow \{ \quad D;C \} \$$
 $\rightarrow \{ d \quad D2 ; C \} \$$
 $\rightarrow \{ d, \quad D ; C \} \$$
 $\rightarrow \{ d, d \quad D2; C \} \$$
 $\rightarrow \{ d, d; \quad C \} \$$
 $\rightarrow \{ d, d; c C2 \} \$$
 $\rightarrow \{ d, d; c \} \$$

Note that it is leftmost derivation

The expression example

1. $E \rightarrow TE'$
2. $E' \rightarrow +TE' | \epsilon$
3. $T \rightarrow FT'$
4. $T' \rightarrow *FT' | \epsilon$
5. $F \rightarrow (E) | \text{int}$



	+	*	(	)	int	\$
E			$E \rightarrow TE'$		$E \rightarrow TE'$	
E'	$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$		$E' \rightarrow \epsilon$
T			$T \rightarrow FT'$		$T \rightarrow FT'$	
T'	$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$		$T' \rightarrow \epsilon$
F			$F \rightarrow (E)$		$F \rightarrow \text{int}$	

Parsing int*int

Stack	Remaining input	Action
E\$	int*int \$	predicate $E \rightarrow TE'$
TE'\$	int*int \$	predicate $T \rightarrow FT'$
FT'E' \$	int*int \$	predicate $F \rightarrow \text{int}$
int T' E' \$	int*int \$	match int
T' E' \$	* int \$	predicate $T' \rightarrow *FT'$
* F T' E' \$	* int \$	match *
F T' E' \$	int \$	predicate $F \rightarrow \text{int}$
int T' E' \$	int \$	match int
T' E' \$	\$	predicate $T' \rightarrow \epsilon$
E' \$	\$	predicate $E' \rightarrow \epsilon$
\$	\$	match \$. success.

Parsing the wrong expression $\text{int}^*]\text{+int}$

Stack	Remaining input	Action
E\$	$\text{int}^*]\text{+int}$ \$	predicate $E \rightarrow TE'$
TE'\$	$\text{int}^*]\text{+int}$ \$	predicate $T \rightarrow FT'$
FT'E' \$	$\text{int}^*]\text{+int}$ \$	predicate $F \rightarrow \text{int}$
int T' E' \$	$\text{int}^*]\text{+int}$ \$	match int
T' E' \$	*]+int \$	predicate $T' \rightarrow *FT'$
* FT' E' \$	*]+ int \$	match *
F T' E' \$	]+int \$	error, skip]
F T' E' \$	+ int \$	PT[F, +] is sync, pop F
T'E'\$	+int\$	predicate $T' \rightarrow \epsilon$
E' \$	+int\$	predicate $E' \rightarrow +TE'$
... ..		

It is easy for LL(1) parsers to skip error

	+	*	(	)	int	]	\$
E			$E \rightarrow TE'$		$E \rightarrow TE'$	error	
E'	$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$		error	$E' \rightarrow \epsilon$
T			$T \rightarrow FT'$		$T \rightarrow FT'$	error	
T'	$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$		error	$T' \rightarrow \epsilon$
F	sync (Follow(F))	sync	$F \rightarrow (E)$	sync	$F \rightarrow \text{int}$	error	sync

Error handling

- There are three types of error processing:
 - report, recovery, repair
- General principles
 - Try to determine that an error has occurred as soon as possible. Waiting too long before declaring an error can cause the parser to lose the actual location of the error.
 - Error report: A suitable and comprehensive message should be reported. “Missing semicolon on line 36” is helpful, “unable to shift in state 425” is not.
 - Error recovery: After an error has occurred, the parser must pick a likely place to resume the parse. Rather than giving up at the first problem, a parser should always try to parse as much of the code as possible in order to find as many real errors as possible during a single run.
 - A parser should avoid *cascading errors*, which is when one error generates a lengthy sequence of spurious error messages.

Error report

- Report an error occurred and what and where possibly the error is;
 - Report expected vs found tokens by filling holes of parse table with error messages)

	+	*	(	)	int	\$
E			$E \rightarrow TE'$	Err, int or (expected in line ...	$E \rightarrow TE'$	
E'	$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$		$E' \rightarrow \varepsilon$
T			$T \rightarrow FT'$		$T \rightarrow FT'$	
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$		$T' \rightarrow \varepsilon$
F			$F \rightarrow (E)$		$F \rightarrow \text{int}$	

Error Recovery

- Error recovery: a single error won't stop the whole parsing. Instead, the parser will be able to resume the parsing at certain place after the error;
 - Give up on current construct and restart later:
 - Delimiters help parser synch back up
 - Skip until find matching `)`, `end`, `]`, whatever
 - Use First and Follow to choose good synchronizing tokens
 - Example:
 - **double** d; use Follow(TYPE) $D \rightarrow \text{TYPE ID SEMI}$
 - **junk** double d; use First(D)
- Error repair: Patch up simple errors and continue .
 - Insert missing token (`;`)
 - Add declaration for undeclared name

Types of errors

- Types of errors
 - Lexical: `@+2`
 - Captured by JLex
 - Syntactical: `x=3+*4;`
 - Captured by parsers like javacup
 - Semantic: `boolean x; x = 3+4;`
 - Captured by type checker, not implemented in parser generators
 - Logical: `infinite loop`
 - Not captured by compilers

Summarize LL(1) parsing

- Massage the grammar
 - Remove ambiguity
 - Remove left recursion
 - Left factoring
- Construct the LL(1) parse table
 - First(), Follow()
 - Fill in the table entry
- Run the LL(1) parse program

Readings in the Textbook

2.4 Parsing	60
– 2.4.1 Top-Down Parsing	61
– 2.4.2 Predictive Parsing	64
– 2.4.3 When to Use ϵ -Productions	65
– 2.4.4 Designing a Predictive Parser	66
– 2.4.5 Left Recursion	67
4.4 Top-Down Parsing	217
– 4.4.1 Recursive-Descent Parsing	219
– 4.4.2 FIRST and FOLLOW	220
– 4.4.3 LL(1) Grammars	222
– 4.4.4 Nonrecursive Predictive Parsing	226